School of Future Tech

Case Study Report

on

Customer Feedback System (C++)

by

Pranav Santhosh Nair

150096725089

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. Reference

1. Introduction to the Case Study

The Customer Feedback System is a console-based application developed using C++ that allows businesses to collect and analyze customer opinions about their services or products. The system provides a simple and structured way for users to submit feedback along with ratings and messages.

The program stores customer feedback data in memory using vectors and structures, enabling efficient storage and retrieval of multiple responses. The application includes features for collecting feedback, displaying individual responses, and calculating the average rating to evaluate overall customer satisfaction.

This project demonstrates the use of basic programming concepts such as structures, loops, vectors, input validation, and functions in C++. The system simulates how businesses can gather customer insights to improve their services and enhance user experience.

2. Problem Statement / Case Background (Abstract)

- **Background**

  Customer feedback is an essential component for improving business services and understanding customer satisfaction levels. However, many small businesses and startups lack simple digital tools for collecting structured feedback from customers.

Traditional feedback methods such as paper forms or unorganized digital comments often result in:

- Poor organization of feedback data
- Difficulty analyzing customer ratings
- Lack of automated summary reports
- Inefficient storage of feedback responses

To solve these problems, a simple feedback management system is required that allows businesses to easily collect, store, and analyze feedback information.

- ## Abstract

This case study presents a Customer Feedback System developed using C++ that allows users to submit feedback including their name, rating, and comments. The system stores feedback entries using structures and vectors, ensuring that multiple responses can be recorded efficiently.

Users interact with the system through a menu-driven interface where they can:

- Submit feedback
- View collected feedback summaries
- Exit the system

The program performs input validation to ensure ratings are entered between 1 and 5, maintaining data accuracy. It also calculates the average customer rating based on all responses received.

This case study demonstrates how fundamental C++ programming techniques can be used to create a simple yet effective data collection and analysis system.

3. Case Study Design

The Customer Feedback System is designed as a menu-driven console application consisting of several interconnected components.

1. User Interface Layout

The program uses a text-based menu interface that allows users to interact with the system easily.

Main menu options include:

1. Give Feedback
2. View Feedback Summary
3. Exit

The system repeatedly displays the menu until the user chooses to exit the program.

2. Core Features

Feedback Collection

Users can submit feedback by entering:

- Customer name
- Rating (1–5)
- Feedback message

The feedback data is stored in a vector.

Feedback Storage

Each feedback entry is stored using a structure (struct) that contains:

- Customer Name
- Rating
- Feedback Message

Feedback Summary Display

The system displays all feedback entries along with:

- Customer name
- Rating
- Message

It also calculates the average rating from all responses.

Input Validation

The program ensures that:

- Rating must be between 1 and 5
- Invalid inputs are rejected

Component Breakdown

Data Structure

A structure named `Feedback` is used to store individual feedback entries.

```
struct Feedback {
  string customerName;
  int rating;
  string message;
};
```

Data Storage

A vector is used to store multiple feedback entries dynamically.

```
vector<Feedback> feedbackList;
```

Functions Used

The program is divided into modular functions:

- `collectFeedback()` → Collects customer responses
- `displaySummary()` → Displays feedback summary
- `displayMenu()` → Displays the main menu

This modular approach improves readability and maintainability.

•

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Several programming techniques and algorithms are used in this system.

• Feedback Storage Algorithm

Each feedback entry is added to the vector.

feedbackList.push_back(fb)

This allows dynamic storage of unlimited feedback responses.

• Average Rating Calculation

The program calculates the overall rating using the formula:

Average Rating=∑RatingsTotal ResponsesAverage\ Rating = \frac{\sum Ratings}{Total\ Responses}Average Rating=Total Responses∑Ratings

Implemented in the code as:

average = totalRating / feedbackList.size();

• Input Validation Algorithm

The program ensures ratings are valid using a **while loop**:

while rating < 1 OR rating > 5

ask user to re-enter rating

This prevents invalid data from being stored.

• Menu Control Algorithm

The application runs continuously using a **do-while loop**.

```
do
  display menu
  perform selected action
while choice != 3
```

This keeps the system active until the user chooses to exit.

• Technology Stack Used for the Case

1. Programming Language

- C++

2. Libraries Used

- `<iostream>` → Input/output operations
- `<string>` → String handling
- `<vector>` → Dynamic storage of feedback
- `<iomanip>` → Formatting output values

3. Programming Concepts Applied

- Structures (`struct`)
- Vectors
- Functions
- Loops
- Input validation
- Menu-driven programming

4. Execution Environment

The program can be executed using:

- CodeBlocks
- Dev C++

- Visual Studio
- GCC compiler

It runs in a **command-line interface (CLI)** environment.

5. Problem Statement / Case Study Implementation Details and Snapshots.

c++ > index.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>

using namespace std;

struct Feedback {
    string customerName;
    int rating;
    string message;
};

void collectFeedback(vector<Feedback>& feedbackList) {
    Feedback fb;

    cout << "\n--- Give Feedback ---\n";
    cout << "Enter your name: ";
    cin.ignore();
    getline(cin, fb.customerName);

    cout << "Enter your rating (1-5): ";
    while (!(cin >> fb.rating) || fb.rating < 1 || fb.rating > 5) {
        cout << "Invalid rating. Please enter a number between 1 and 5: ";
        cin.clear();
        cin.ignore(1000, '\n');
    }

    cout << "Enter your feedback message: ";
    cin.ignore();
    getline(cin, fb.message);

    feedbackList.push_back(fb);
    cout << "\nThank You for Your Feedback!\n";
}

void displaySummary(const vector<Feedback>& feedbackList) {
    if (feedbackList.empty()) {
        cout << "\nNo feedback has been collected yet.\n";
        return;
    }

```

c++ > index.cpp > main()

```cpp
void displaySummary(const vector<Feedback>& feedbackList) {

    cout << "\n--- Feedback Summary ---\n";
    double totalRating = 0;

    for (int i = 0; i < feedbackList.size(); i++) {
        cout << "\nFeedback #" << i + 1 << ":\n";
        cout << "  Customer : " << feedbackList[i].customerName << "\n";
        cout << "  Rating   : " << feedbackList[i].rating << " / 5\n";
        cout << "  Message  : " << feedbackList[i].message << "\n";
        totalRating += feedbackList[i].rating;
    }

    double average = totalRating / feedbackList.size();
    cout << "\n----------------------------------------\n";
    cout << "Total Responses : " << feedbackList.size() << "\n";
    cout << fixed << setprecision(1);
    cout << "Average Rating  : " << average << "\n";
    cout << "----------------------------------------\n";
}

void displayMenu() {
    cout << "\n============================\n";
    cout << "  Customer Feedback System\n";
    cout << "============================\n";
    cout << "1. Give Feedback\n";
    cout << "2. View Feedback Summary\n";
    cout << "3. Exit\n";
    cout << "Enter your choice: ";
}

```

```cpp
int main() {
    vector<Feedback> feedbackList;
    int choice;

    cout << "Welcome to the Customer Feedback System!\n";

    do {
        displayMenu();

        while (!(cin >> choice)) {
            cout << "Invalid input. Please enter a number: ";
            cin.clear();
            cin.ignore(1000, '\n');
        }

        switch (choice) {
            case 1:
                collectFeedback(feedbackList);
                break;
            case 2:
                displaySummary(feedbackList);
                break;
            case 3:
                cout << "\nThank you for using the Customer Feedback System. Goodbye!\n";
                break;
            default:
                cout << "\nInvalid choice. Please select 1, 2, or 3.\n";
        }

    } while (choice != 3);

    return 0;
}
```

```
cd "/Users/pnair/Desktop/midsem-2/c++/" && g++ index.cpp -o index && "/Users/pnair/Desktop/midsem-2/c++/"index
pnair@Pranavs-MacBook-Air-3 midsem-2 % cd "/Users/pnair/Desktop/midsem-2/c++/" && g++ index.cpp -o index && "/Users/pnair/Desktop/midsem-2/c++/"index
Welcome to the Customer Feedback System!

============================
  Customer Feedback System
============================
1. Give Feedback
2. View Feedback Summary
3. Exit
Enter your choice: 1

--- Give Feedback ---
Enter your name: Pranav Nair
Enter your rating (1-5): 5
Enter your feedback message: very good

Thank You for Your Feedback!
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

The Customer Feedback System successfully demonstrates:

- Efficient feedback collection using structured data
- Dynamic storage using vectors
- Accurate calculation of average customer ratings
- Reliable input validation for rating values

The system allows businesses to easily review feedback and understand customer satisfaction levels.

Conclusion

This case study presents a simple yet effective Customer Feedback System built using C++. The application demonstrates core programming concepts including structures, vectors, loops, and functions.

The system provides a clear framework for collecting and analyzing customer feedback in a structured format. While the current version is console-based, the system can be expanded in the future to include:

- File storage for saving feedback
- Graphical user interface (GUI)
- Database integration
- Web-based feedback forms

This makes the project an ideal learning example for students studying C++ programming and software development fundamentals.

- Real-time fitness APIs
- User authentication
- Database integration
- Team competitions
- Mobile application versions

This makes FitTrack Pro an ideal academic case study for ITM Skills University projects focused on user-centered web application design.

7. References

- C++ Programming Documentation – ISO C++ Standard
- C++ STL Documentation (Vectors and Strings)
- Tutorials on menu-driven programming in C++
- Online resources and guides for input validation and data structures
- Academic references on customer feedback systems and survey data collection